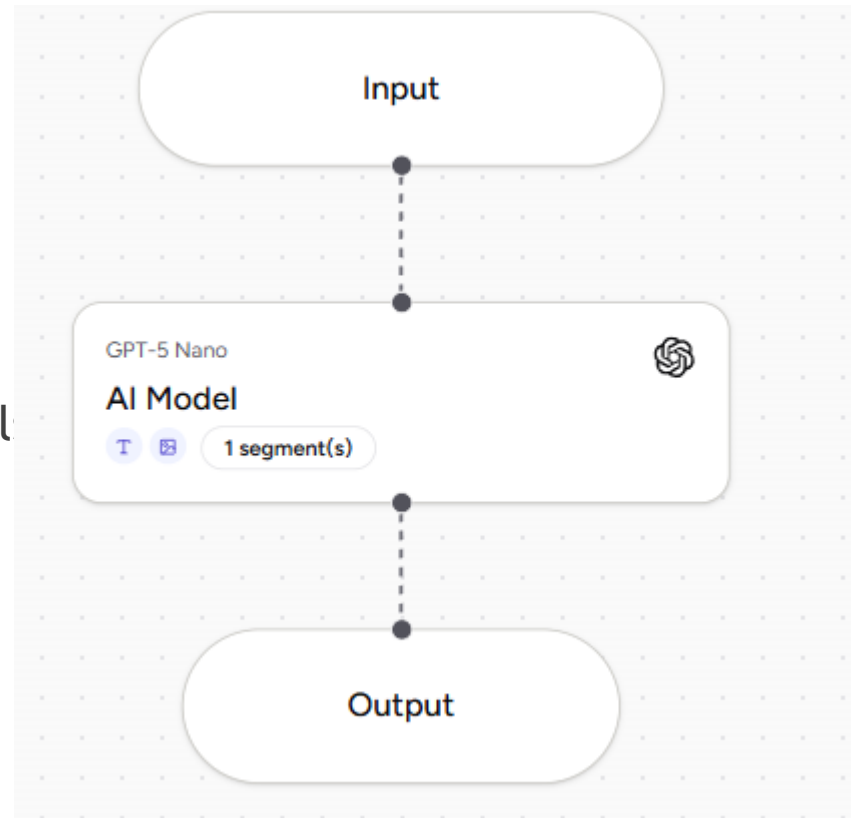


SOC Analyst AI Agent

By: Asim Siddiqui

1. Build the AI Agent

- ▶ Sign up to airia.ai and create your own project
- ▶ Add your own AI model and build four agent
- ▶ Publish the agent and get the API detail including the key



2. Run the Python code

- In this step, we will write a Python automation script and run it on the internal server. This script will run a packet capture in the background. The aim is to find malicious IPs and send those in the form of JSON alerts to AIRIA using the API.

```
import subprocess
import csv
import json
import os
import uuid
import requests
from collections import Counter

# ----- CONFIGURATION -----
# CONFIGURATION

INTERFACE = "eth0" # Change if needed (check with: ip a)
CAPTURE_DURATION = 100 # 5 minutes
THRESHOLD = 40 # Packet threshold

PCAP_FILE = "traffic.pcap"
CSV_FILE = "traffic.csv"
ALERT_FILE = "alert.json"

# ----- Airia Webhook -----
AIRIA_API_URL = "https://api.airia.ai/v2/PipelineExecution/12a76fca-1"
AIRIA_API_KEY = "ak-MTE50Tk3ODM2OHwxNzc5OTg2NDg5NzkxfHRpLVFXVjBaWEp1"

# Metadata
DESTINATION_HOST = "Internal-server"
DESTINATION_IP = "10.211.85.94"

# ----- HELPER -----
def run_command(cmd, description):
    print(f"[+] {description}")
    subprocess.run(cmd, check=True)

# STEP 1 - Capture Traffic
```

3. Attack the Server

Python Code Flow

Perform a traffic capture

Convert it to CSV using tshark

Find offending IPs based on threshold **> 40**

Convert data to JSON and send to AIRIA API

From the attacker machine generate some malicious traffic. For this lab, I have used ping only for demonstration purposes. You can try different types of DOS attacks. While the traffic is generated, the packet count will increment on the Internal server as per the logic built into the code.

```
Parrot Terminal
File Edit View Search Terminal Help
-[parrot@parrot]~]
$ping 10.211.85.94 -c 50
PING 10.211.85.94 (10.211.85.94) 56(84) bytes of data.
4 bytes from 10.211.85.94: icmp_seq=1 ttl=64 time=1.88 ms
4 bytes from 10.211.85.94: icmp_seq=2 ttl=64 time=0.880 ms
4 bytes from 10.211.85.94: icmp_seq=3 ttl=64 time=1.07 ms
4 bytes from 10.211.85.94: icmp_seq=4 ttl=64 time=0.667 ms
4 bytes from 10.211.85.94: icmp_seq=5 ttl=64 time=0.525 ms
4 bytes from 10.211.85.94: icmp_seq=6 ttl=64 time=1.06 ms
4 bytes from 10.211.85.94: icmp_seq=7 ttl=64 time=0.768 ms
4 bytes from 10.211.85.94: icmp_seq=8 ttl=64 time=0.799 ms
4 bytes from 10.211.85.94: icmp_seq=9 ttl=64 time=0.756 ms
4 bytes from 10.211.85.94: icmp_seq=10 ttl=64 time=0.828 ms
4 bytes from 10.211.85.94: icmp_seq=11 ttl=64 time=0.674 ms
4 bytes from 10.211.85.94: icmp_seq=12 ttl=64 time=0.797 ms
4 bytes from 10.211.85.94: icmp_seq=13 ttl=64 time=1.00 ms
4 bytes from 10.211.85.94: icmp_seq=14 ttl=64 time=0.798 ms
4 bytes from 10.211.85.94: icmp_seq=15 ttl=64 time=0.678 ms
4 bytes from 10.211.85.94: icmp_seq=16 ttl=64 time=1.06 ms
4 bytes from 10.211.85.94: icmp_seq=17 ttl=64 time=0.919 ms
4 bytes from 10.211.85.94: icmp_seq=18 ttl=64 time=0.988 ms
4 bytes from 10.211.85.94: icmp_seq=19 ttl=64 time=0.777 ms
4 bytes from 10.211.85.94: icmp_seq=20 ttl=64 time=0.616 ms
4 bytes from 10.211.85.94: icmp_seq=21 ttl=64 time=1.57 ms
```

4. Airia AI analysis

Step 4: AIRIA AI analysis

01

AIRIA API Receives the JSON

02

AIRIA Agent performs actions
based on SOC playbook

03

AIRIA Agent sends back the
analysis to the Internal server

5. Result

It detected the attacker IP as malicious

```
(kali@kali)-[~]
└─$ python3 new_soc_capture.py
[+] Capturing on eth0 for 100s
Capturing on 'eth0'
123
[+] Capture saved to traffic.pcap
[+] CSV created at traffic.csv

[+] Traffic volume per source IP:
10.211.85.80: 123 packets

[!] Suspicious IP detected: 10.211.85.80
[+] Alert JSON written to alert.json
[+] Sending alert to Airia Agent Execution API ...
[+] Airia responded with status 200
[+] Airia Response JSON:
{
  "type": "string",
  "result": "{\n  \"alert_id\": \"SOC-DB9FA68DA\", \n  \"threat_classification\": \"Unknown\", \n  \"risk_score\": 0, \n  \"risk_level\": \"Low\", \n  \"confidence_level\": \"Low\", \n  \"mitre_mapping\": {\n    \"tactic\": \"\", \n    \"technique_id\": \"\", \n    \"technique_name\": \"\", \n    \"analysis_reasoning\": \"Input payload is missing required fields: source_host and protocol. Without source_host, protocol, and a confirmed internal/external context, a reliable risk assessment and MITRE mapping cannot be performed. Available data indicates 123 packets over a 100-second window from 10.211.85.80 toward 10.211.85.94 within the internal scope, but this alone is insufficient to classify the activity.\", \n    \"recommended_actions\": [\n      \"Monitor\", \n    ], \n    \"escalation_required\": false, \n    \"executive_summary\": \"The alert cannot be adequately evaluated due to missing essential fields. Provide a complete payload to determine if this is benign traffic or something requiring action.\"\n  }, \n  \"report\": null, \n  \"isBackupPipeline\": false, \n  \"executionId\": \"25ace52c-4ad0-47e3-bf98-10ec403aca78\", \n  \"userInputId\": \"05d0c8e0-1ea7-4da1-9111-2c7b1cae97d2\", \n  \"files\": [], \n  \"images\": []\n}"
}

[+] Workflow complete.

(kali@kali)-[~]
└─$
```